Dockerize, Push to Docker Hub & Deploy on Kubernetes (AWS EC2)

Deployment Report — Node.js Application

Module 12 Assignment · Batch 13

GitHub Repository	github.com/Byttie/docker-kube
Docker Hub Image	hub.docker.com/repository/docker/rag1b/kube
Docker Hub Tag	rag1b/kube:1.2
Cloud Provider	AWS EC2 (t3.medium, ap-south-1, Ubuntu 26.04 LTS)
Orchestration	Kubernetes (kubeadm, single-node cluster, v1.30.14)
Namespace	production

1. Objective

This project takes a Node.js application from a GitHub repository through a full containerization and orchestration pipeline: building a Docker image, publishing it to Docker Hub, provisioning a Kubernetes cluster on an AWS EC2 instance, and deploying the application into a dedicated `production` namespace, exposed via a NodePort Service.

- Dockerize the application using a custom Dockerfile
- Push the built image to a public Docker Hub repository
- Provision and configure a single-node Kubernetes cluster on an AWS EC2 t3.medium instance
- Create a production namespace and deploy the application into it
- Expose the application externally via a NodePort Service and verify accessibility

2. Repository & Image Links

GitHub Repository (Dockerfile + manifests)	https://github.com/Byttie/docker-kube
Docker Hub Image	https://hub.docker.com/repository/docker/rag1b/kube/general

3. Dockerizing the Application

The application was containerized with a lightweight Alpine-based Dockerfile. The image installs dependencies, copies the source, and starts the app on port 5000.

Dockerfile

```
FROM node:18-alpine
WORKDIR /CICD-OSTAD
COPY . .
RUN npm install
EXPOSE 5000
CMD ["npm", "start"]
```

Building the Image

The image was built locally using `docker build -t rag1b/kube:1.0 .` All four build stages completed successfully and the image was tagged for Docker Hub.

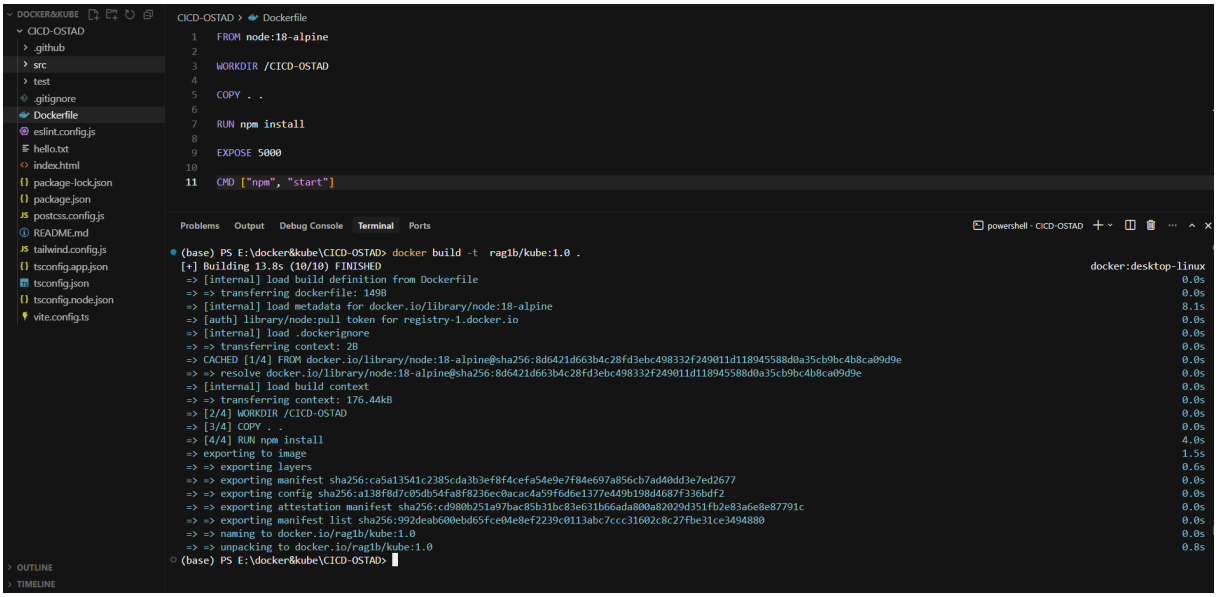


Figure 1 — Docker image build completing successfully (docker build -t rag1b/kube:1.0)

Testing the Container Locally

Before pushing, the image was run locally with port 3000 mapped to the container's port 5000 (`docker run -d -p 3000:5000 --name kube rag1b/kube:1.0`) to confirm the application starts correctly.

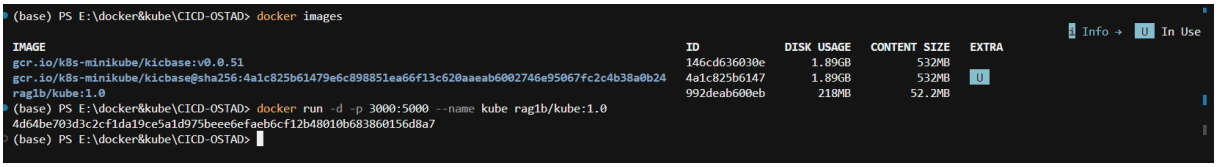


Figure 2 — Local images list and container run confirmation

4. Pushing the Image to Docker Hub

The tagged image was pushed to the public Docker Hub repository `rag1b/kube` using `docker push rag1b/kube:1.0`. After that due to some bugs, the image was rebuild and pushed in the same repo using different tags (1.1 and 1.2). `kube:1.2 is the final build.` All layers were pushed or mounted successfully and a manifest digest was returned, confirming a successful publish.

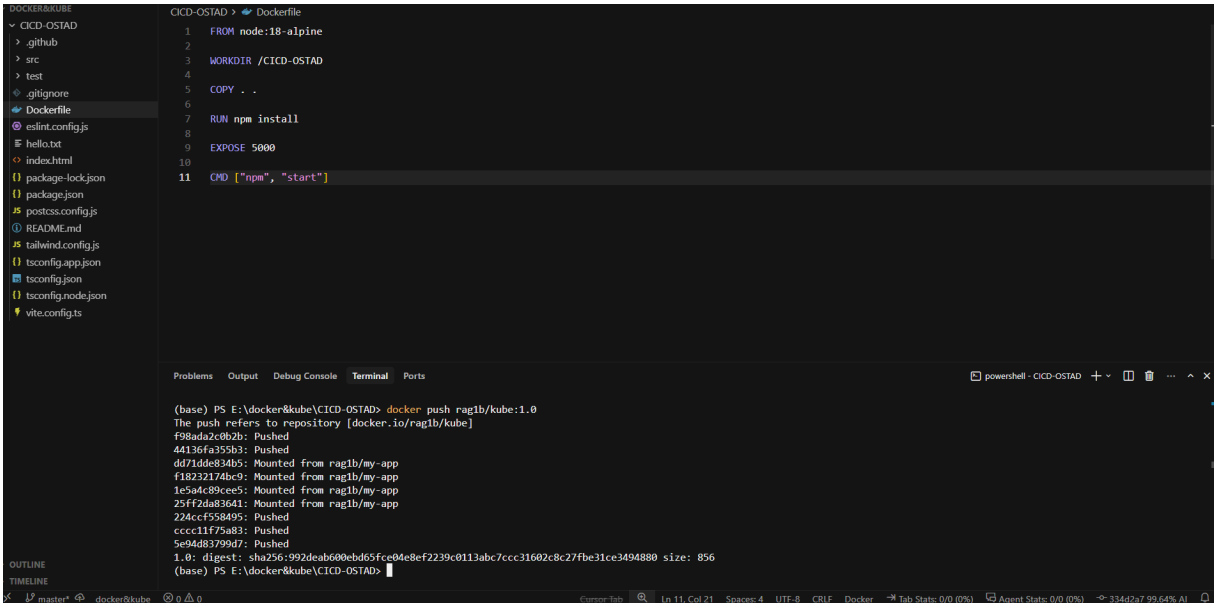


Figure 3 — docker push output showing all layers pushed successfully

The Docker Hub repository page confirms the `1.2` tag is live, publicly listed under the `rag1b/kube` repository.

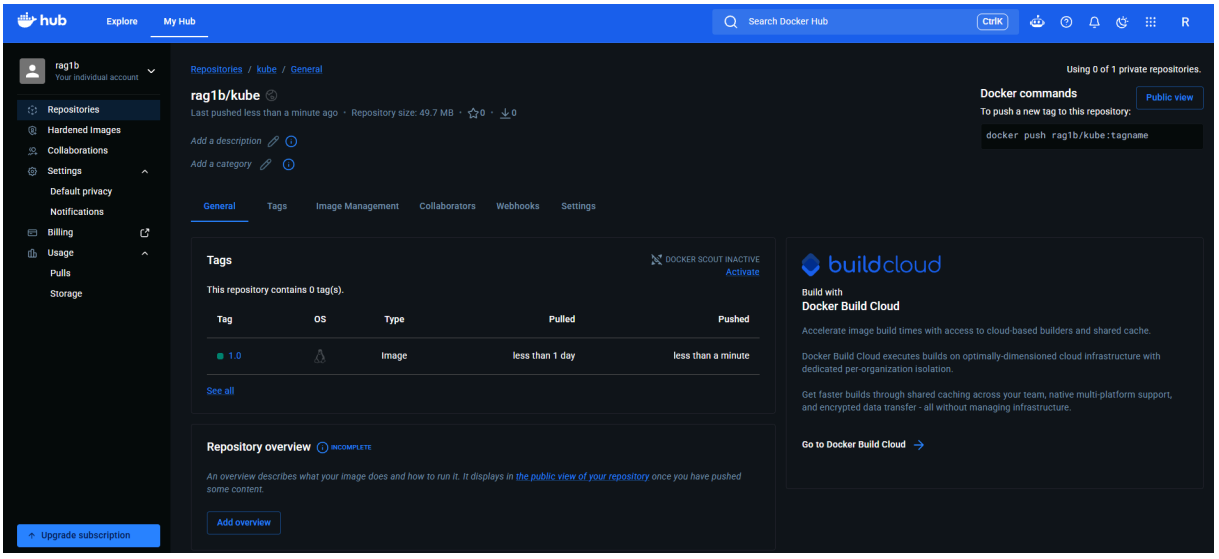


Figure 4 — Docker Hub repository page showing the pushed 1.0 tag

5. Provisioning the AWS EC2 Instance

A t3.medium EC2 instance running Ubuntu 26.04 LTS was launched in the ap-south-1 (Mumbai) region. The instance's security group was configured to allow the ports required for Kubernetes and remote administration:

Port / Range	Protocol	Purpose
22	TCP	SSH access
6443	TCP	Kubernetes API server
30000 – 32767	TCP	NodePort service range



Figure 5 — EC2 instance summary: t3.medium, Running, with inbound rules for ports 22, 6443 and 30000–32767

Connecting to the Instance

The instance was accessed via SSH using its key pair. The connection confirms Ubuntu 26.04 LTS is running with low resource usage prior to installing Kubernetes components.

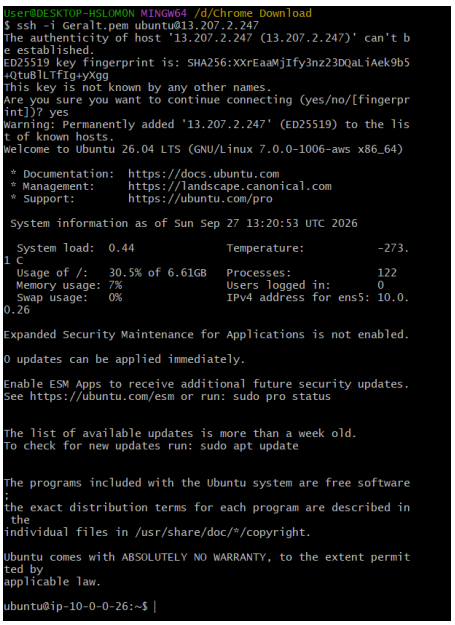


Figure 6 — SSH connection into the EC2 instance (Ubuntu 26.04 LTS)

6. Kubernetes Cluster Setup

On the EC2 instance, Docker, kubeadm, kubelet and kubectl were installed, and a single-node Kubernetes cluster was initialized with `kubeadm init`, followed by the Flannel CNI plugin for pod networking. Once the cluster was active, kubectl was configured to communicate with it.

Cluster status was later verified with `kubectl get nodes`, confirming the control-plane node `ip-10-0-0-26` was Ready on Kubernetes v1.30.14 (see Figure 10, Section 9).

7. Creating the Production Namespace

A dedicated namespace named `production` was created to isolate the application's resources from the cluster's default namespaces.

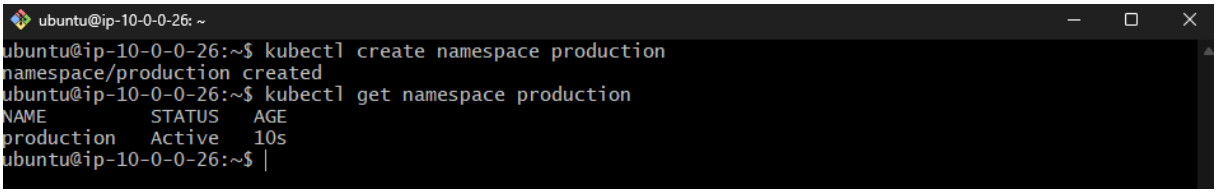


Figure 7 — production namespace created and verified as Active

8. Deploying the Application

A single manifest file (`deployment.yaml`) defines both the Deployment and the Service, and was applied to the `production` namespace:

Deployment

Runs **2 replicas** of the app using the `rag1b/kube:1.2` image pulled from Docker Hub, listening on container port **4000**.

Service

Type **NodePort**, mapping service port 4000 to the container's port 4000, and exposing it externally on node port **30080**.

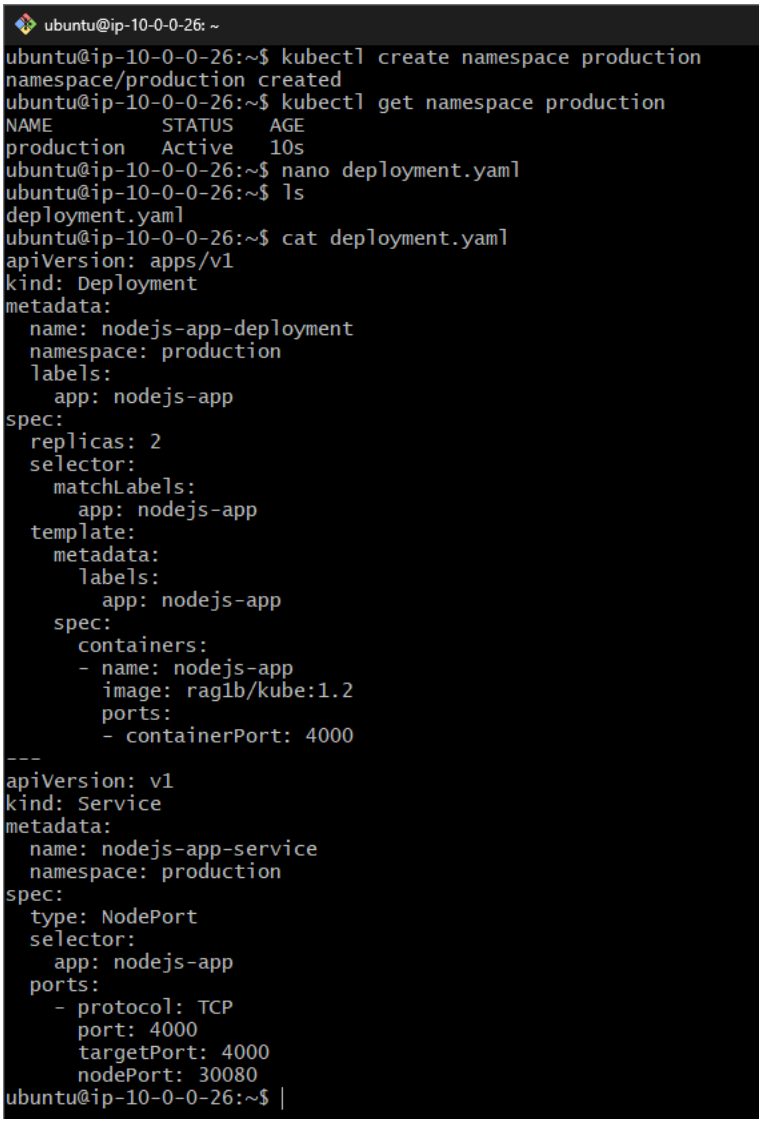


Figure 8 — deployment.yaml defining the Deployment (2 replicas, rag1b/kube:1.2) and the NodePort Service (port 30080)

9. Verification

After applying the manifest, the cluster and application state were verified end-to-end:

- **Namespaces** — the production namespace is Active alongside the default kube-system namespaces.
- **Nodes** — the control-plane node ip-10-0-0-26 is Ready on Kubernetes v1.30.14.
- **Pods** — both nodejs-app-deployment replicas are Running (1/1 Ready, 0 restarts).
- **Connectivity** — a curl request to http://13.207.2.247:30080 returns the application's HTML page.
- **Browser access** — opening the NodePort URL in a browser renders the “Welcome / Simanta Kumar Roy” page, confirming the service is reachable from outside the cluster.

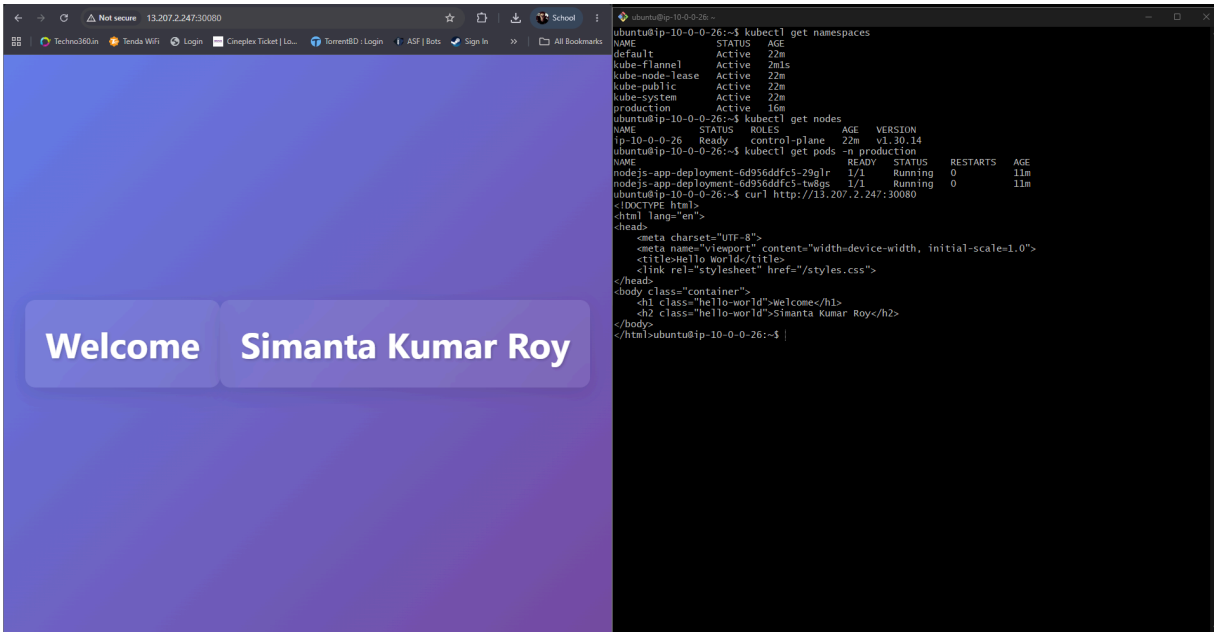


Figure 9 — kubectl verification (namespaces, nodes, pods Running) alongside the live application in the browser at 13.207.2.247:30080

For comparison, Figure 10 below shows the same application rendered locally at `localhost:4000` prior to Kubernetes deployment — the Kubernetes-served output at NodePort 30080 (Figure 9) is visually identical, confirming a consistent deployment.

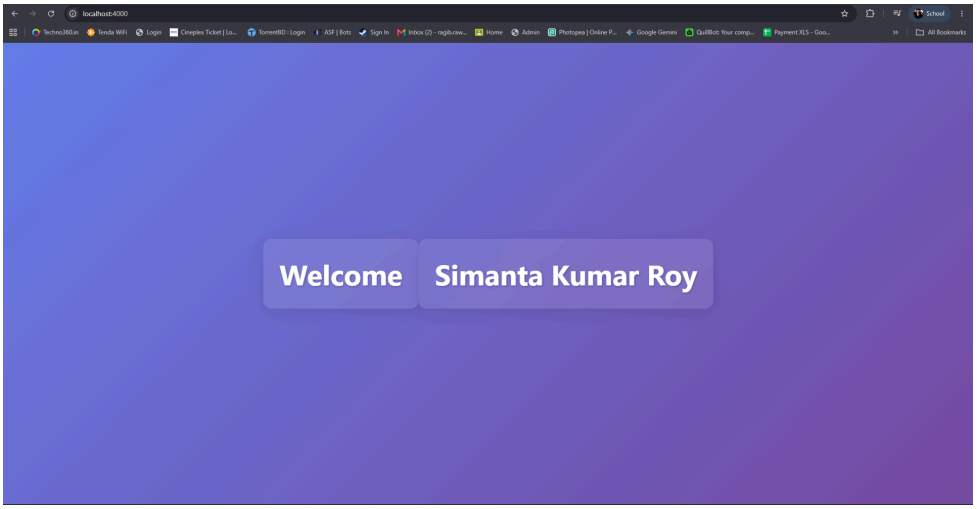


Figure 10 — Application running locally at localhost:4000 before deployment

10. Conclusion

The Node.js application was successfully containerized, published to Docker Hub as `rag1b/kube`, and deployed onto a self-managed single-node Kubernetes cluster running on an AWS EC2 t3.medium instance. The application is running with 2 replicas in the `production` namespace and is publicly reachable via a NodePort Service on port 30080, satisfying all assignment requirements.